NEOLABS

SECURITY LABS | SOC LEVEL 1

Week 02 - The Ghost Login

Weekly Learning Pack | Learn + Connect + Operate

Programme: NeoLabs x Renaissance Innovation Labs Cybersecurity Internship
Week 02
Version: 1.0
Classification: Student Training Material - Authorised Synthetic Use Only
Authorised synthetic training use only.

Why this week matters

A suspicious-login report is one of the most common ways a security investigation begins. A user may report an unfamiliar sign-in, a strange notification, a failed login they do not remember, or activity at an unexpected time. The analyst's job is not to immediately label every unusual event as an attack. The analyst must collect evidence, understand normal behaviour, build a timeline and decide what the evidence supports.

Week 2 introduces the SOC habit that will be used throughout the internship: **baseline first, evidence second, conclusion last**.

It also introduces an important operational relationship: **IT Security Support receives the user's problem first, while SOC independently validates the security story from telemetry.** A good support handoff gives SOC a precise account, time window and user context. SOC then proves or disproves the security concern in Wazuh and returns a defensible conclusion.

Learning outcomes

By the end of this pack you should be able to:

- prepare the supported Ubuntu EC2 SOC workstation and launch the NeoLabs Wazuh stack;
- authenticate the workstation to the server-assigned pod using your private NeoLabs Access Code;
- locate the Wazuh dashboard URL and local dashboard credentials safely;
- explain the difference between authentication, authorisation and session activity;
- recognise useful fields in authentication telemetry;
- distinguish ordinary login mistakes from suspicious patterns;
- build a chronological authentication timeline;
- correlate multiple events without relying on one alert alone;
- use a Support escalation as an investigation lead without treating it as ground truth;
- record evidence in a way another analyst can reproduce;
- decide when an event is benign, suspicious or a confirmed incident inside the lab story;
- propose a practical Wazuh detection improvement;
- validate that a fixed scenario no longer produces the same unsafe behaviour.

1. Set up your SOC workstation on Ubuntu EC2

For SOC interns using a remote workstation, the supported path is an **Ubuntu or Debian Linux instance**. The examples below use Ubuntu on AWS EC2.

1.1 Launch the instance

Use Ubuntu 22.04 LTS or Ubuntu 24.04 LTS. The current SOC toolkit expects enough capacity for Docker and Wazuh:

- 8 GiB RAM is the preferred minimum;
- 12-16 GiB RAM is recommended when available;
- at least 25 GiB free disk is required, with 50 GiB recommended.

Keep your private SSH key secure. Never commit it to GitHub or upload it to the assignment repository.

1.2 Configure the EC2 Security Group

For this controlled internship workstation, the simplest lab rule is:

```
Type: All TCP
Source: My IP / your approved current public IPv4 address (/32)
```

This allows your own workstation to reach the ports the lab may use while keeping the instance restricted to your source address.

Important: Do not set the source to `0.0.0.0/0` or `Anywhere-IPv4`. Opening all TCP ports to the whole Internet is not required for this internship and is not a safe production practice. At minimum, SSH TCP 22 and the Wazuh dashboard TCP 8443 must be reachable from your approved source IP.

If your public IP changes, update the Security Group source before troubleshooting the NeoLabs launcher.

1.3 SSH into the instance

From a terminal that has your EC2 private key:

```
ssh -i "your-key.pem" ubuntu@<EC2-PUBLIC-IP>
```

If your local operating system requires it, make the key readable only by you before connecting:

```
chmod 400 your-key.pem
```

1.4 Update Ubuntu and install Git

Run:

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y git
```

1.5 Clone the SOC toolkit

```
git clone https://github.com/NeoLabs-Security/RIL_NeoLabs-SOC1-interns-toolkit-.git
cd RIL_NeoLabs-SOC1-interns-toolkit-
```

If you already cloned the repository earlier, update it instead:

```
git pull origin main
```

1.6 Run the one-click SOC launcher

From the toolkit root:

```
bash start-neolabs-soc.sh
```

Do **not** run the whole launcher with `sudo`.

On the first successful run, the launcher checks or prepares the supported native Linux runtime, including Docker Engine, Docker Compose v2, the Wazuh indexer kernel requirement, local Wazuh credentials/certificates, the pinned Wazuh stack, NeoLabs authentication, the assigned-pod telemetry connection and the Wazuh telemetry pipeline.

When prompted, enter:

1. your assigned pod number; and
2. your private NeoLabs Access Code.

The pod is still enforced server-side. Editing local files does not change your authorised pod.

Wait until you see:

```
SOC WORKSTATION READY
```

READY means assigned-pod synthetic telemetry has been processed and is searchable in Wazuh - not merely that Docker containers started.

1.7 Reprint your Wazuh dashboard URL and credentials

After the first setup, you can run the same launcher without attempting to open a browser on the remote server:

```
bash start-neolabs-soc.sh --no-browser
```

On a supported headless Ubuntu EC2 instance, the launcher reports the dashboard addresses it can determine, including a public URL when EC2 metadata is available. It also reports:

```
Username: admin
Password: <your private locally generated dashboard password>
```

Typical remote dashboard format:

```
https://<EC2-PUBLIC-IP>:8443
```

Do not post terminal screenshots that contain the dashboard password.

1.8 Confirm your pod and current scenario

From the toolkit root:

```
bash neolabs status
bash neolabs pod info
```

These commands show your current lab state, scenario and server-assigned pod.

The current SOC CLI intentionally does **not** expose a separate learner-application target through `bash neolabs targets`; SOC target selection is hidden so telemetry remains the primary SOC surface. If the Week 2 assignment requires you to open the VCC learner application directly, use only the application URL published by the mentor/current assignment for your pod. Do not guess a hostname or construct another pod's URL from its number.

For workstation or telemetry problems, run:

```
bash neolabs doctor
```

2. Authentication telemetry

Authentication telemetry is the evidence produced when a system verifies identity. In the VCC lab this may include application logs, API events and Wazuh-ingested telemetry. Useful fields commonly include:

Field	Why it matters
Event time	Establishes sequence and duration
User/account	Identifies the synthetic identity involved
Result	Successful, failed, blocked or challenged login
Source address	Helps compare origin patterns inside the authorised lab
User agent/device	Helps distinguish expected and unexpected client behaviour
Event type	Tells you whether the record represents login, logout, failure or another auth action
Request/session identifier	Helps correlate related events
Correlation identifier	Links events that belong to the same investigation chain

Field	Why it matters
Failure reason	May distinguish a typo, disabled account or controlled suspicious activity

An individual field rarely proves the whole story. Analysts gain confidence by correlating several fields across time.

Analyst note: An unfamiliar source address is an indicator to investigate, not automatic proof of compromise. Context still matters.

3. Baseline before anomaly

A baseline is a picture of expected behaviour. Before investigating the suspicious period, review normal synthetic login activity for your assigned pod.

Ask:

- Which accounts normally authenticate?
- What does a normal successful login look like?
- What does a normal password mistake look like?
- Are there predictable retries?
- Which fields remain stable across ordinary activity?
- Which events are generated by the application itself versus a learner action?

When you understand the baseline, anomalies become easier to explain.

4. Patterns that deserve attention

Inside this controlled exercise, investigate patterns such as:

- repeated failed attempts followed by a success;
- an account appearing from an unexpected lab source;
- several synthetic accounts showing a similar pattern in a short period;
- success events with no expected preceding user behaviour;
- unusual timing compared with the baseline window;
- a mismatch between the user's report and the event sequence;
- an account-state condition that does not match the observed authentication outcome.

Do not assume all of these are malicious. Your report should state what the evidence shows and what remains uncertain.

5. Build a timeline

A useful authentication timeline is concise and chronological.

Example structure:

Time (UTC)	Account	Event	Source	Analyst interpretation	Evidence reference
10:03:11	learner-a	Login failed	lab-source	Possible typo	EV-01
10:03:19	learner-a	Login success	same source	Consistent with retry	EV-02
10:18:42	learner-b	Login success	different source	Requires correlation	EV-03

Use the timestamps from the telemetry. Do not rewrite a timeline from memory after the investigation.

Evidence requirement: Every important conclusion in your final report should point back to a screenshot, event identifier, redacted log excerpt or other approved evidence reference.

6. Wazuh investigation workflow

After your SOC workstation is READY, confirm your lab context:

```
bash neolabs status
bash neolabs pod info
```

Your local Wazuh environment should receive only the telemetry associated with your server-assigned pod.

A beginner-friendly investigation sequence is:

1. Confirm the assigned time window from the GitHub Issue or Support escalation.
2. Review the normal baseline period.
3. Filter for authentication-related events.
4. Use the account, reported window or correlation fields from the case as starting pivots.
5. Compare failed and successful authentication events.
6. Correlate authentication, session and relevant support/application events.
7. Add relevant evidence to your register.
8. Build the timeline.
9. State the most likely explanation and your confidence.
10. Escalate or return findings only when the evidence meets the exercise threshold.

7. How IT Security Support and SOC work together

Week 2 is intentionally cross-functional. The same suspicious-login story can enter the organisation through the Support desk and then become a SOC investigation.

Support owns the user-facing intake

IT Security Support should:

- receive the synthetic user's report;
- verify the user's identity using the approved procedure;
- preserve the reported timestamp and relevant device/browser context;
- record whether the user remembers the login;

- record the current access/account state that Support is authorised to view;
- avoid destructive recovery until required evidence is preserved; and
- send SOC a focused escalation when the case is suspicious.

SOC owns the telemetry-based security conclusion

SOC should:

- treat the Support ticket as an investigation lead, not ground truth;
- independently search Wazuh for the reported account/time window;
- correlate authentication, session, application and support events;
- follow `request_id`, `correlation_id`, session identifiers and event time where useful;
- classify the evidence as benign, suspicious, confirmed within the lab scenario, or insufficient evidence;
- recommend the next security action or request additional facts from Support; and
- preserve detection/reporting evidence separately from Support's recovery notes.

A strong Support-to-SOC handoff includes:

```
Ticket/case reference
Synthetic account reference
Reported time window
User confirms / denies / is unsure about the login
Approved device/browser context
Current access/account state
Evidence already preserved
Actions already taken by Support
Specific question for SOC to answer
```

A useful SOC question is neutral, for example:

> Please correlate authentication and session activity for this synthetic account during the reported window and determine whether the observed sign-in is consistent with normal user activity.

Do **not** accept a statement such as "this is the Ghost Login" as proof. The SOC analyst must still establish the conclusion from telemetry.

Closing the loop

After SOC reaches a conclusion, the result should inform Support's next authorised step:

```
User report -> Support triage -> SOC telemetry investigation ->
SOC finding/recommendation -> authorised Support recovery -> validation/closure
```

This relationship mirrors real security operations: Support understands the user-facing symptom; SOC provides security visibility and correlation.

8. Avoid common analyst mistakes

Alert equals incident

An alert is a signal to investigate. It is not the conclusion.

Support escalation equals incident

A user's report can be important, but it is still a report. Validate it against telemetry.

One strange field equals attacker

A single unusual field may have an innocent explanation. Correlate it.

Collect everything

More evidence is not automatically better. Collect enough to support or challenge your hypothesis while respecting privacy and scope.

Change the system while investigating

SOC Level 1 analysts should not erase, modify or 'clean up' evidence. Preserve first, then follow the approved escalation/containment process.

Forget the time zone

Record the time zone used in your timeline. Prefer UTC when the assignment does not specify otherwise.

9. Classification language

Use precise language:

Benign: Evidence is consistent with expected behaviour or a normal user mistake.

Suspicious: Evidence contains meaningful anomalies, but available information does not yet prove the controlled incident story.

Confirmed within the lab scenario: Multiple correlated evidence points satisfy the exercise's defined threshold.

Insufficient evidence: The available records do not support a reliable conclusion yet.

Avoid dramatic wording. A professional SOC report explains what happened, what supports the conclusion and what should happen next.

10. Detection improvement thinking

Week 2 also introduces detection engineering at a beginner level. You are not expected to build an advanced production rule. Instead, identify a useful signal that would help analysts notice the same pattern sooner.

A good proposal answers:

- What event fields are required?
- What pattern should trigger attention?
- Over what time window?
- Which normal behaviour might create false positives?
- What severity is appropriate?
- What should the analyst check before escalating?

Example concept: 'Several failed authentications for the same synthetic account followed by a successful authentication within a short lab window.'

The exact thresholds should come from the exercise context rather than being treated as universal production values.

11. Retesting the fixed scenario

A retest is not simply 'the alert disappeared.' Confirm that:

- the intended authentication behaviour still works;

- the previously suspicious/unsafe path no longer behaves the same way;
- useful telemetry still reaches Wazuh;
- the fix did not remove the evidence analysts need;
- your detection proposal still makes sense after the change.

Record the fixed-version evidence separately from the original investigation.

12. Your Week 2 operating sequence

```
1. Provision/update the Ubuntu SOC workstation.
2. Clone/update the SOC toolkit.
3. Run bash start-neolabs-soc.sh and authenticate with pod + Access Code.
4. Wait for SOC WORKSTATION READY.
5. Use bash start-neolabs-soc.sh --no-browser when you need the dashboard URL/credential again.
6. Confirm pod/scenario with bash neolabs status and bash neolabs pod info.
7. Read the Week 2 GitHub Issue and Support handoff, if one is assigned.
8. Baseline normal authentication behaviour in Wazuh.
9. Correlate the reported account/time window with authentication and session telemetry.
10. Build the timeline and evidence register.
11. Classify only what the evidence supports.
12. Write the requested report/detection proposal and return a useful finding to Support.
13. Retest when the fixed scenario is released.
14. Submit through the central assignment repository Pull Request workflow.
```

Safety boundary

- Do not attempt to select another pod.
- Do not test real accounts or systems.
- Do not commit Access Codes, session tokens, Wazuh credentials, certificates, SSH keys or private URLs.
- Do not expose the EC2 instance to all Internet sources merely for convenience.
- Do not change/delete evidence to make the timeline cleaner.
- If telemetry appears to contain another cohort member's real information, stop and notify a mentor.

Quick knowledge check

1. Why should the EC2 Security Group use your approved source IP instead of `0.0.0.0/0`?
2. What does `SOC WORKSTATION READY` prove?
3. Which commands confirm your current NeoLabs state and assigned pod?
4. Why is a Support escalation an investigation lead rather than proof of compromise?
5. Name four fields that should be included in a Support-to-SOC handoff.
6. Why is a baseline useful before investigating an unfamiliar login?
7. What should you verify during a fixed-version retest besides whether an alert disappeared?

Remember

Prepare the workstation correctly. Baseline first. Correlate the Support lead with telemetry. State only what the evidence supports.